

⚠ **Security Warning:** Work in an isolated environment (virtual machine or container). Never open unknown files on your primary machine.

Challenge Description

Additional details will be available after launching your challenge instance.

Hints

- Look at the problem category
-

Tools Used

- `wget` – Download files from a web server
 - `file` – Determine file type
 - `strings` – Extract printable character sequences
 - `exiftool` – Read image metadata
 - `binwalk` – Detect embedded files and hidden data
 - `zsteg` – Detect steganography in PNG/BMP images
-

Complete Solution

Step 1: Launch the Challenge Instance

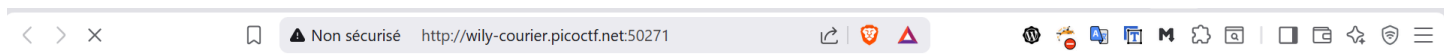
After launching the challenge instance, you are given a URL:

```
http://wily-courier.picoctf.net:50271/
```

Note: The port number may change each time you launch the instance.

Step 2: Explore the Website

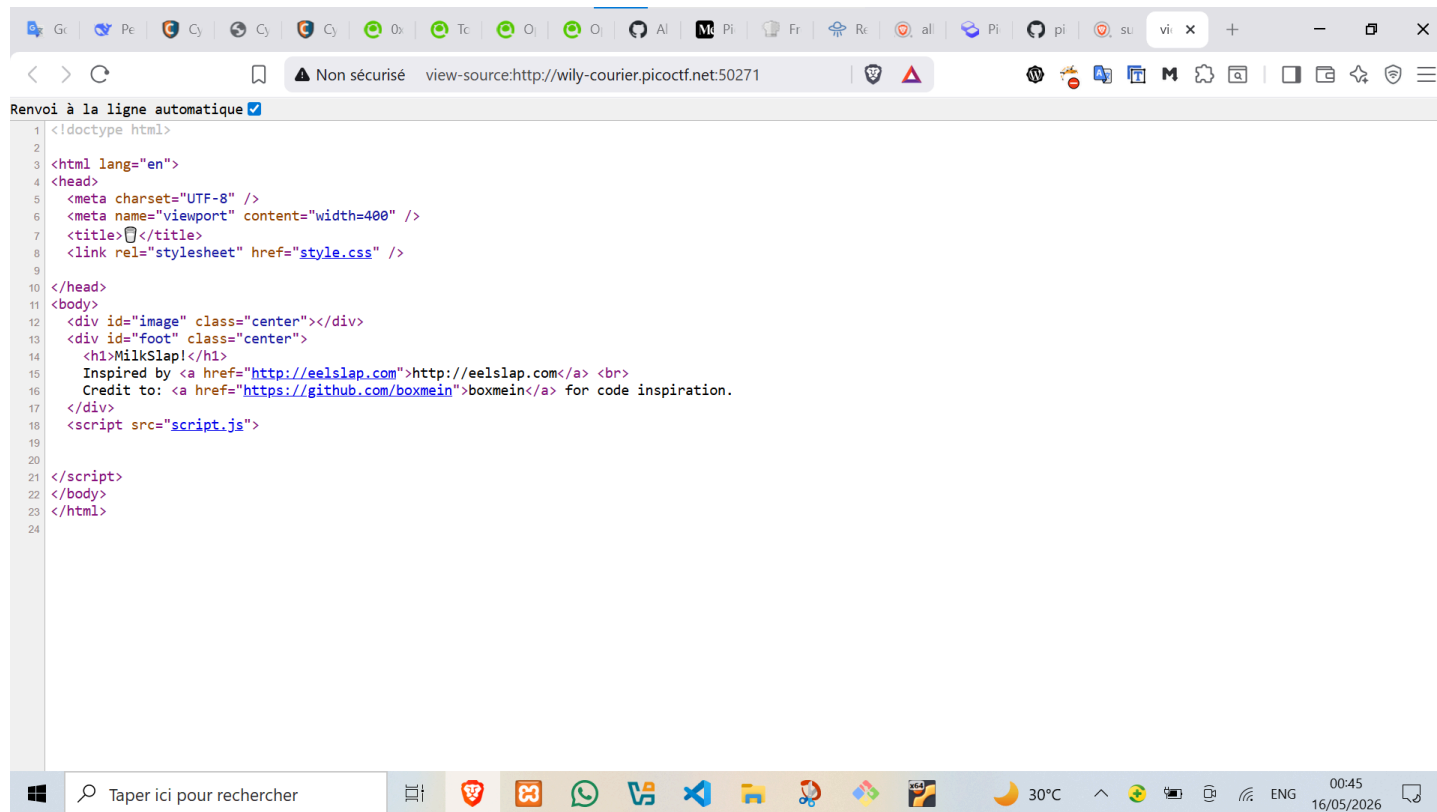
Open the URL in a browser:



The page displays a simple interface with a milk cup image. Clicking the image doesn't reveal anything obvious.

Step 3: Check the Page Source

View the page source (right-click → "View Page Source" or `Ctrl+U`):



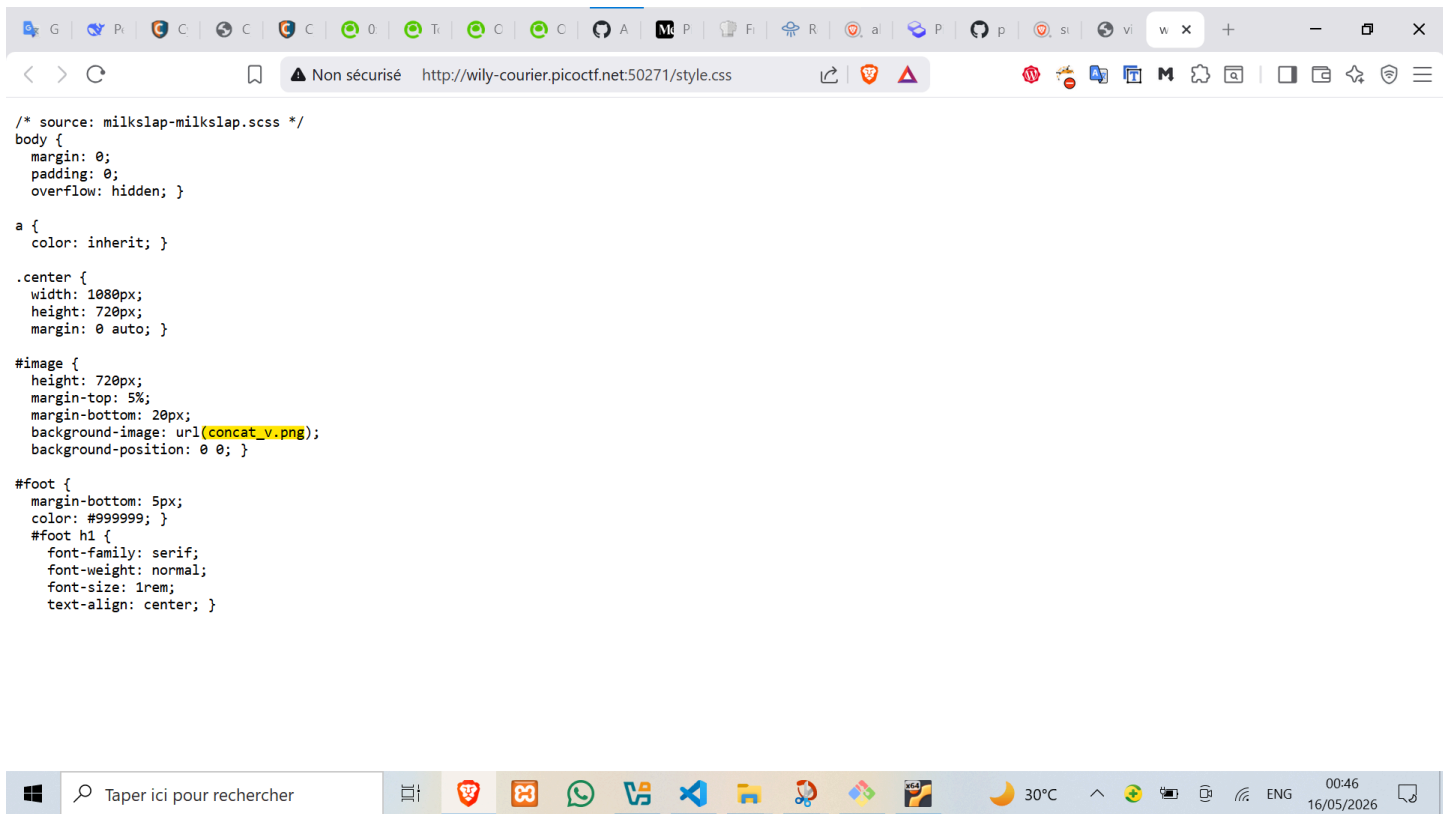
Nothing suspicious appears in the HTML source.

Step 4: Inspect the CSS

Looking deeper, we examine the linked CSS file. One of the CSS rules catches our attention:

```
#image {  
    height: 720px;  
    margin-top: 5%;  
    margin-bottom: 20px;  
    background-image: url(concat_v.png);  
    background-position: 0 0;  
}
```

CSS



Key finding: There's an image URL inside the CSS: `concat_v.png`

Step 5: Download the Hidden Image

Download the image using `wget` or directly from the browser:

```
wget http://wily-courier.picoctf.net:50271/concat_v.png
```

Step 6: Basic Image Inspection

First, check the image file type and dimensions:

```
file concat_v.png
```

Output:

```
concat_v.png: PNG image data, 1280 x 47520, 8-bit/color RGB, non-interlaced
```

Interesting observation: The image has a **height of 47,520 pixels** – unusually tall! This suggests the image might contain stitched-together data.

Step 7: Search for Visible Text

Check if the flag appears as plain text:

```
strings concat_v.png | grep -i "picoCTF"
```

Output: (no results)

Step 8: Examine Metadata

Use `exiftool` to inspect metadata:

```
exiftool concat_v.png
```

Output:

```
ExifTool Version Number      : 13.50
File Name                    : concat_v.png
Directory                   : .
File Size                    : 18 MB
File Modification Date/Time   : 2026:05:15 18:41:32+02:00
File Access Date/Time        : 2026:05:15 18:42:11+02:00
File Inode Change Date/Time   : 2026:05:15 18:42:08+02:00
File Permissions              : -rw-rw-rw-
File Type                    : PNG
File Type Extension           : png
MIME Type                    : image/png
Image Width                   : 1280
Image Height                  : 47520
Bit Depth                     : 8
Color Type                    : RGB
Compression                   : Deflate/Inflate
Filter                        : Adaptive
Interlace                     : Noninterlaced
Image Size                    : 1280x47520
Megapixels                    : 60.8
```

Nothing suspicious in the metadata.

Step 9: Scan with Binwalk

Check for embedded files:

```
binwalk concat_v.png
```

Output:

DECIMAL	HEXADECIMAL	DESCRIPTION

41	0x29	Zlib compressed data, default compression
1364499	0x14D213	JBOOT SCH2 kernel header, compression type: none, Entry Point: 0xC911A155, image size: 4113974432 bytes, data CRC: 0x3030F6F0, Data Address: 0xD792122B, rootfs offset: 0xE683325B, rootfs size: 4142915508 bytes, rootfs CRC: 0xE9B020D0, header CRC: 0x5510C4D2, header size: 62453 bytes, cmd line length: 60903 bytes
3210141	0x30FB9D	MySQL ISAM compressed data file Version 2
11738439	0xB31D47	JBOOT STAG header, image id: 6, timestamp 0xD1410605, image size: 2284567946 bytes, image JBOOT checksum: 0x9345, header JBOOT checksum: 0x9211

Nothing obvious. The image contains some embedded JBOOT headers, but not the flag.

Step 10: Analyze with zsteg (LSB Steganography)

Since the image is very tall, it might contain LSB-hidden data. However, `zsteg` crashes with a stack overflow error:

```
zsteg concat_v.png
```

Error:

```
/var/lib/gems/3.3.0/gems/zpng-0.4.6/lib/zpng/scan_line.rb:369:in `prev_scanline_byte':
stack level too deep (SystemStackError)
...
```

Step 11: Increase Ruby Stack Size

The error occurs because the image is so tall that it causes a recursion depth issue in the `zsteg` library. To fix this, increase the Ruby thread VM stack size:

```
export RUBY_THREAD_VM_STACK_SIZE=500000000
```

Now run `zsteg` again:

```
zsteg concat_v.png
```

Output:

```
imagedata          .. text: "\n\n\n\n\n\n\t\t"
chunk:0:IHDR        .. file: Adobe Photoshop Color swatch, version 0, 1280 colors; 1st
RGB space (0), w 0xb9a0, x 0x802, y 0, z 0; 2nd HSB space (1), w 0, x 0, y 0, z 0
b1,b,lsb,xy        .. text: "picoCTF{<REDACTED>}\n"
b1,bgr,lsb,xy      .. <wbStego size=0x941a5b ext=nil
data="\xB6\xAD\xB6}\xDB\xB2lR\x7F\xDF\x86\xB7c\xFC\xFF\xBF\x02Zr\x8E\xE2Z\x12\xD8q\xE5
&MJ-
X:\xB5\xBF\xF7\x7F\xDB\xDFI\bm\xDB\xDB\x80m\x00\x00\x00\xB6m\xDB\xDB\xB6\x00\x00\x00\x
B6\xB6\x00m\xDB\x12\x12m\xDB\xDB\x00\x00\x00\x00\x00\xB6m\xDB\x00\xB6\x00\x00\x00\xDB\
xB6mm\xDB\xB6\xB6\x00\x00\x00\x00\x00m\xDB" even=true hdr=nil enc=nil mix=true
controlbyte="[">
b2,r,lsb,xy        .. text: ["U" repeated 8 times]
b2,r,msb,xy        .. file: VISX image file
b2,g,lsb,xy        .. file: VISX image file
b2,g,msb,xy        .. file: SoftQuad DESC or font file binary - version 15722
b2,b,msb,xy        .. text: "UfUUUU@UUU"
b4,r,lsb,xy        .. text: "\"\"\"\"\"\"#4D"
b4,r,msb,xy        .. text: "www3333"
b4,g,lsb,xy        .. text: "wewwwwvUS"
b4,g,msb,xy        .. text: "\"\"\"\"\"DDDD"
b4,b,lsb,xy        .. text: "vdUeVwweDFw"
b4,b,msb,xy        .. text: "UUYYUUUUUUUU"
```

Key finding: The line `b1,b,lsb,xy` reveals the flag!

```
picoCTF{<REDACTED>}
```

Final Result

picoCTF{<REDACTED>}

Note: The actual flag is redacted here. In the real challenge, running `zsteg` with the increased stack size will reveal the complete flag.

Key Concepts

Concept	Description
Web Inspection	Checking page source and CSS files can reveal hidden resources.
Image Steganography	Data can be hidden in the least significant bits (LSB) of image pixels.
zsteg	A tool for detecting LSB steganography in PNG and BMP images.
Stack Overflow	Very large images can cause recursion depth issues in image processing libraries.
Ruby Stack Size	Increasing <code>RUBY_THREAD_VM_STACK_SIZE</code> fixes recursion errors with large images.
Image Dimensions as Clue	An unusually tall image (47,520 pixels) suggests stitched or hidden data.

Pro Tips

- Always check CSS files** – Hidden resources like images can be referenced in CSS `background-image` properties.
- Large images are suspicious** – If an image has unusual dimensions (extremely tall or wide), it may contain stitched-together data or hidden information.

3. **Fix `zsteg` errors** – When you encounter `stack level too deep`, increase the Ruby stack size with `export RUBY_THREAD_VM_STACK_SIZE=500000000`.
4. **Check all bit planes** – The flag was found in `b1,b,lsb,xy` (blue channel, least significant bit). Data can be in any channel and any bit plane.
5. **Don't ignore errors** – The `zsteg` error was a clue that the image is unusually large and requires special handling.